Team 7

ScheduleSkies
Software Design Document
Version 01.00

Revision History

Date	Version	Description	Author
02/26/2026	01.00	Initial Creation	Labiste, Medalla, Pescadero, Susan, Villamor

Table of Contents

1. INTRODUCTION 4
   - 1.1 Purpose 4
   - 1.2 Scope 4
   - 1.3 Overview 4
   - 1.4 Reference Material 4
   - 1.5 Definitions and Acronyms 4
2. SYSTEM OVERVIEW 5
3. SYSTEM ARCHITECTURE 5
   - 3.1 Architectural Design 5
   - 3.2 Decomposition Description 7
   - 3.3 Design Rationale 8
4. DATA DESIGN 8
   - 4.1 Data Description 8
   - 4.2 Data Dictionary 8
5. COMPONENT DESIGN 10
6. HUMAN INTERFACE DESIGN 11
   - 6.1 Overview of User Interface 11
   - 6.2 Screen Images 12
   - 6.3 Screen Objects and Actions 13
7. REQUIREMENTS MATRIX 14
8. APPENDICES 14

Software Design Document

1. INTRODUCTION

1.1 Purpose

The purpose of this **Software Design Document (SDD)** is to identify and describe the architecture and system design of **ScheduleSkies**. It serves as a detailed guide for developers and designers, providing a blueprint for the system's technical implementation based on the requirements defined in the SRS. The intended audience includes the project development team, stakeholders, and system architects.

1.2 Scope

ScheduleSkies is a smart web-based travel and event planning application. The software focuses on analyzing environmental conditions, such as **weather forecasts** and **traffic predictions**, to assist users (travelers, tourists, and event planners) in choosing optimal times and routes. The primary goal is to provide a unified platform that reduces inefficient planning and scheduling conflicts by integrating real-time environmental data with user preferences and location awareness.

1.3 Overview

This document is organized into several key sections to provide a comprehensive technical view:

- **System Overview**: A general description of functionality and design context.
- **System Architecture**: Detailed breakdown of the layered design pattern and subsystem interactions.
- **Data Design**: Modeling the relational database structure and data dictionary.
- **Component & Interface Design**: Details on internal logic and user interface objects.

1.4 Reference Material

The design is based on the following sources of information:

- ScheduleSkies Software Requirements Specification (SRS) v1.0.
- ScheduleSkies Project Overview.
- ScheduleSkies Feature List

1.5 Definitions and Acronyms

Term	Definition
AI	Artificial Intelligence, utilized for suggested itinerary improvements and travel plan generation.
GPS	Global Positioning System, used for real-time location tracking.
Itinerary	A daily or multi-day schedule or timeline for events and travel plans.
Weather Suitability	A rating system (e.g., 1-5) for determining if outdoor activities are appropriate based on current forecasts.

2. SYSTEM OVERVIEW

Schedule Skies is a specialized web application designed to empower travelers with data-driven itinerary planning. Unlike traditional scheduling tools, Schedule Skies acts as a "weather-aware" personal assistant, helping users anticipate and navigate the environmental variables that often disrupt travel plans specifically rain, crowd density, and traffic congestion.

3. SYSTEM ARCHITECTURE

3.1 Architectural Design

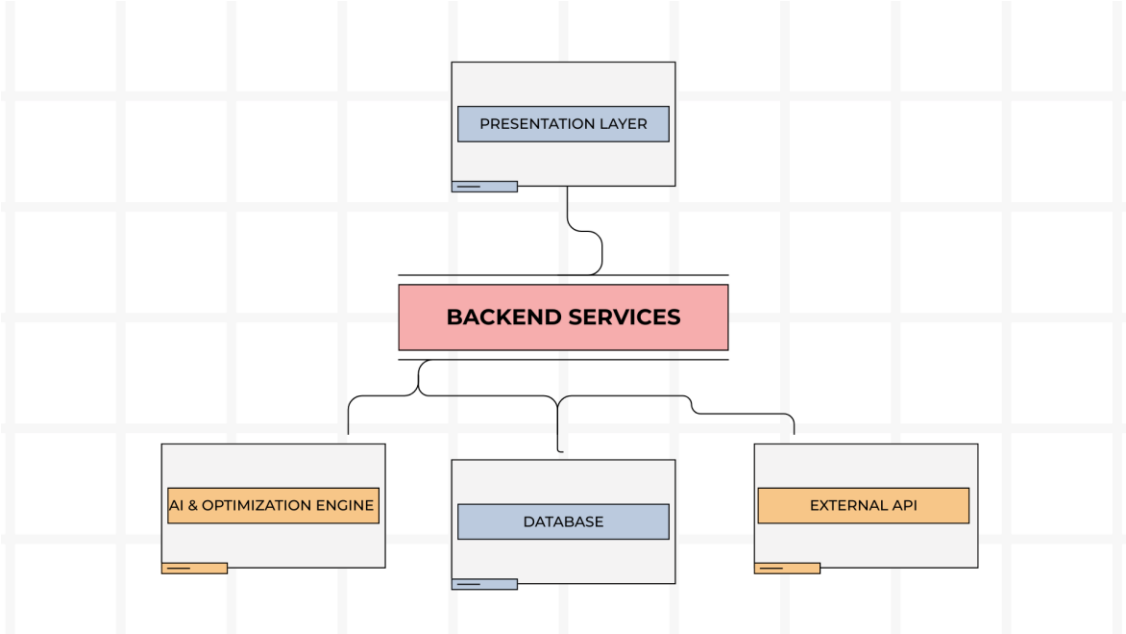
ScheduleSkies adopts a modular, service-oriented web architecture following a layered design pattern. The system is decomposed into independent but collaborative subsystems to ensure scalability, maintainability, and real-time responsiveness.3

The architecture consists of:

1. Presentation Layer (Frontend Web Application)
2. Application Layer (Backend Services / API Layer)
3. AI & Optimization Engine
4. Integration Layer (External APIs)
5. Data Layer (Database & Storage)

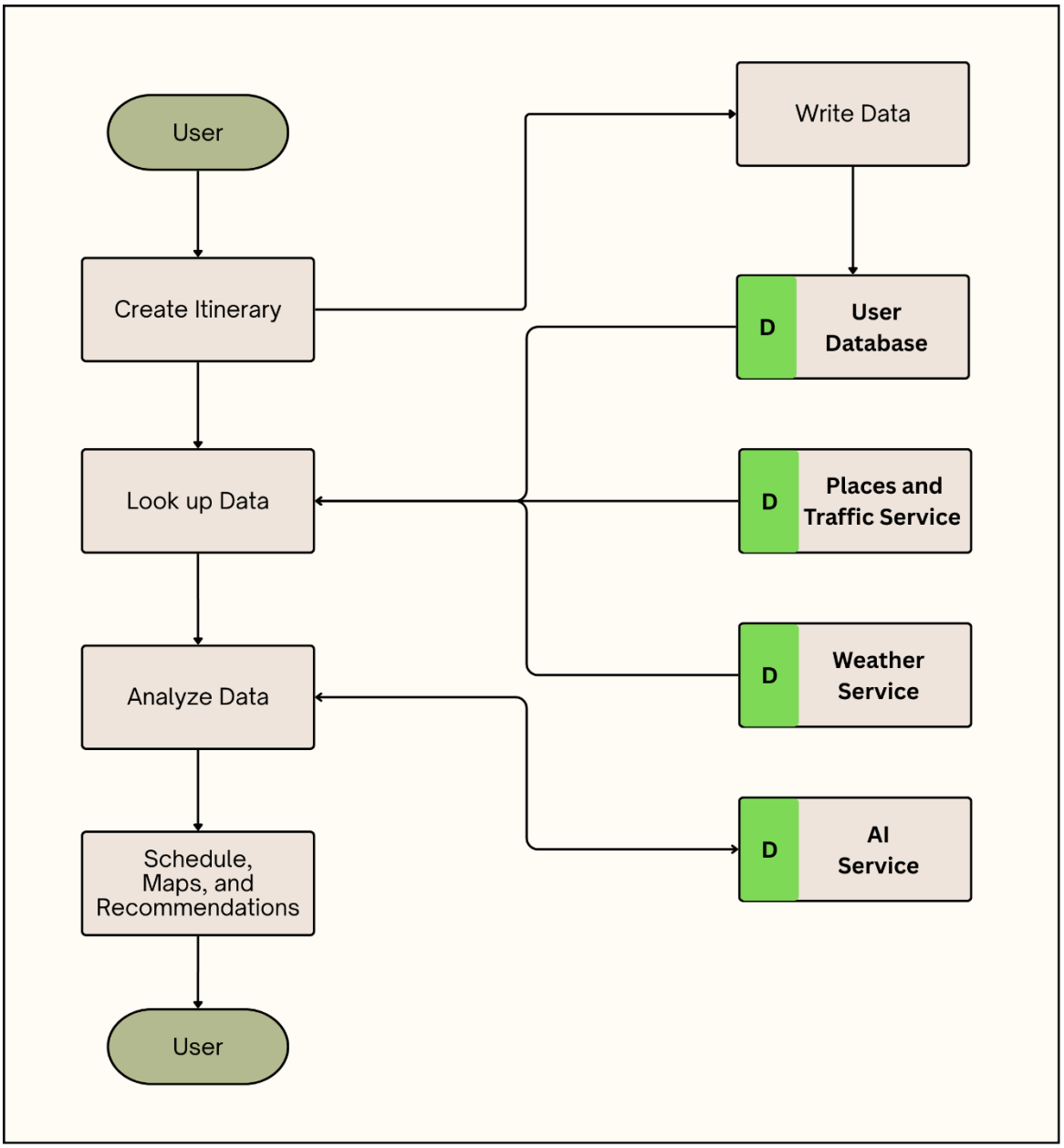
This modular decomposition ensures separation of concerns:

- UI logic is isolated from business logic.
- Business logic is separated from data access.
- External services are abstracted through integration modules.
- AI optimization is handled independently.



The Presentation Layer serves as the user interface where users create itineraries, view maps, and receive alerts, sending all requests to the Application Layer (Backend Services), which acts as the central coordinator of the system. The backend processes business logic such as schedule management, authentication, and collaboration, while retrieving and storing persistent data through the Data Layer (Database & Storage). When intelligent decisions are required, such as optimizing schedules based on weather or traffic, the backend invokes the AI & Optimization Engine, which analyzes itinerary data and generates improved recommendations. To obtain real-time weather, traffic, routing, and location information, the backend communicates with the Integration Layer (External APIs), which securely handles third-party service interactions and returns structured data. Together, these five layers collaborate in a structured flow where the frontend interacts only with the backend, the backend orchestrates intelligence and integrations, the AI engine enhances decision-making, external APIs supply live data, and the database ensures reliable storage and retrieval of system information.

3.2 Decomposition Description



1. The User submits itinerary data, profile information, and search queries to the ScheduleSkies System.
2. The system stores and retrieves persistent information from:
   - User Database
3. The system requests:
   - Weather data from Weather Service
   - Traffic and routing data from Maps Service
   - Location data from Places Service
   - Recommendations from AI Service
4. The system returns:

- Optimized schedules
- Alerts and notifications
- Maps and recommendations

3.3 Design Rationale

The selected modular layered architecture for ScheduleSkies was chosen to ensure clear separation of concerns, scalability, and maintainability while supporting real-time data integration and intelligent optimization. By dividing the system into presentation, application services, AI processing, integration, and data management layers, responsibilities are clearly partitioned, reducing complexity and allowing independent development and future scaling of critical components such as the optimization engine and external API integrations. This approach improves security by centralizing external communications within the backend and enhances reliability through controlled data access and caching mechanisms. A monolithic architecture was considered but rejected due to limited scalability and higher maintenance risk, while a full microservices architecture was deemed unnecessarily complex and resource-intensive for Version 1.0. The chosen design balances performance, extensibility, development feasibility, and future growth potential.

4. DATA DESIGN

4.1 Data Description

The system's information domain is modeled using a relational database structure to enforce data integrity. The core entities are Users, Itineraries, Events, and Locations.

- User data is transformed into structured profiles containing preferences and travel history.
- The Itineraries entity acts as a parent to multiple `Events`, structured in a one-to-many relationship.
- Real-time data (like current weather or traffic delays) is not permanently stored but cached in a transient data store (like Redis) to reduce API call overhead and improve performance. Data persistence relies on Supabase’s automated backup.

4.2 Data Dictionary

Alphabetically list the system entities or major data along with their types and descriptions. If you provided a functional description in Section 3.2, list all the functions and function parameters. If you provide an OO description, list the objects and its attributes, methods, and method parameters.

1. Entity: Event

Attribute	Data Type	Description
event_id	UUID	Primary Key. Unique identifier for the event.
itinerary_id	UUID	Foreign Key linking the event to its parent Itinerary.
location_id	UUID	Foreign Key linking the event

		to a specific Location.
title	String	The name or description of the activity.
start_time	Timestamp	The scheduled start date and time of the event.
end_time	Timestamp	The scheduled end date and time of the event.
weather_suitability	Integer	A rating (e.g., 1-5) represents the weather suitability rating for outdoor activities.
travel_time_next	Integer	Estimated travel time (in minutes) to the next event in the timeline.

2. Entity: Itinerary

Attribute	Data Type	Description
itinerary_id	UUID	Primary Key. Unique identifier for the itinerary.
owner_id	UUID	Foreign Key linking to the User who created the plan.
title	String	The name of the trip or schedule (e.g., "Paris Vacation").
start_date	Date	The starting date of the travel plan.
end_date	Date	The concluding date of the travel plan.
is_group_plan	Boolean	A flag indicating if this is utilized for group planning and collaborative real-time updates.

3. Entity: Location

Attribute	Data Type	Description
location_id	UUID	Primary Key. Unique identifier for the place.
name	String	The name of the destination or establishment.
latitude	Decimal	GPS coordinate utilized for real-time location tracking.
longitude	Decimal	GPS coordinate utilized for real-time location tracking.

place_type	String	Category of the location (e.g., restaurant, hotel, attraction).
rating	Decimal	The average user rating of the place.

4. Entity: User

Attribute	Data Type	Description
user_id	UUID	Primary Key. Unique identifier managed by Supabase Authentication.
username	String	The user's chosen display name.
email	String	The user's contact email address.
travel_preferences	JSONB	Stores user-specific travel preferences including indoor/outdoor preference, budget, and pace.
saved_locations	Array(UUID)	A list of references to the user's favorite locations.

5. COMPONENT DESIGN

In this section, we take a closer look at what each component does in a more systematic way. Below is a procedural description language pseudocode of each major component identified in Section 3.2. Each function includes a brief description of local data where relevant.

Function: **SubmitItineraryData()**
Purpose: Accept itinerary input from the user and forward it to the system.
Local Data:
itineraryData (list of events)
validationStatus (boolean)
PDL:

```
FUNCTION SubmitItineraryData()
   RECEIVE itineraryData from user
   validationStatus ← ValidateItinerary(itineraryData)

   IF validationStatus = TRUE THEN
      CALL StoreItinerary(itineraryData)
      CALL OptimizeSchedule(userID)
   ELSE
      DISPLAY error message
   ENDIF
END FUNCTION
```

Function: **StoreItinerary(itineraryData)**
PDL:

```
FUNCTION StoreItinerary(itineraryData)
   CONNECT to database
   SAVE itineraryData
   CLOSE connection
END FUNCTION
```

Function: **RetrieveItinerary(userID)**
PDL:

```
FUNCTION RetrieveItinerary(userID)
   CONNECT to database
   FETCH itinerary WHERE userID = userID
   RETURN itinerary
END FUNCTION
```

Function: **GetRecommendations(userID, itinerary)**
Purpose: Provide personalized suggestions.
PDL:

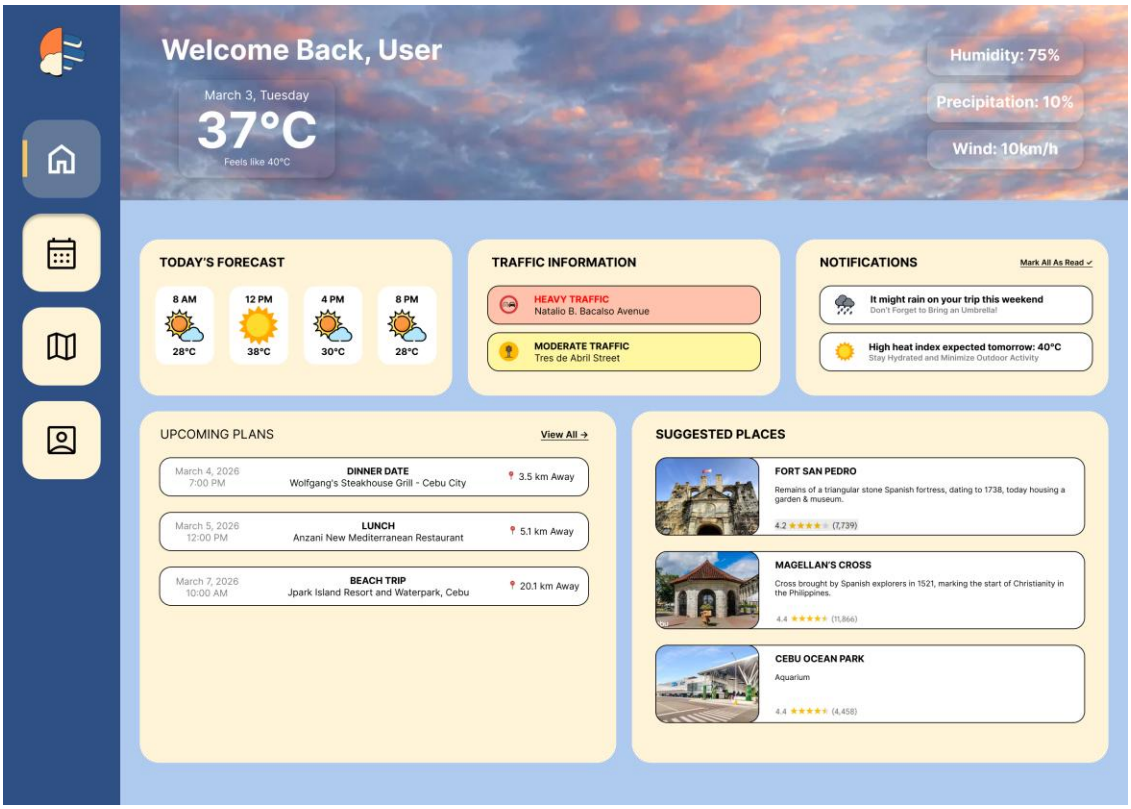
```
FUNCTION GetRecommendations(userID, itinerary)
   SEND user preferences and itinerary to AI Service
   RECEIVE recommendations
   RETURN recommendations
END FUNCTION
```

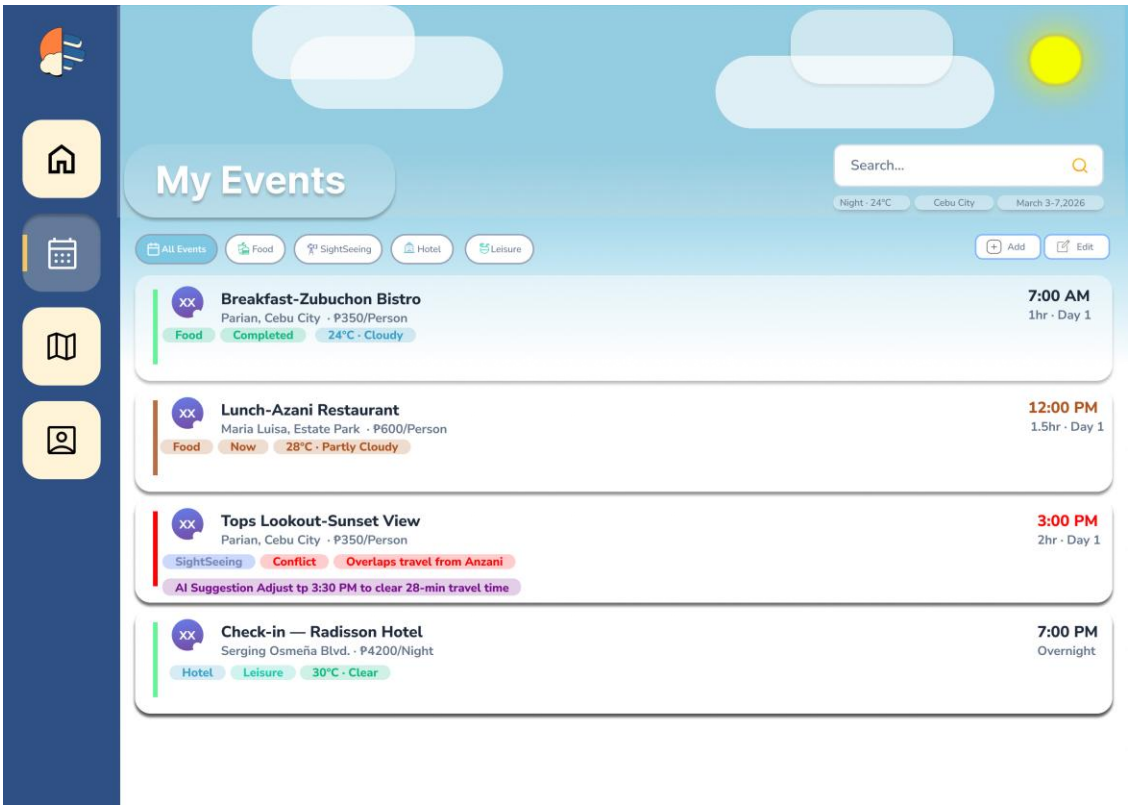
6. HUMAN INTERFACE DESIGN

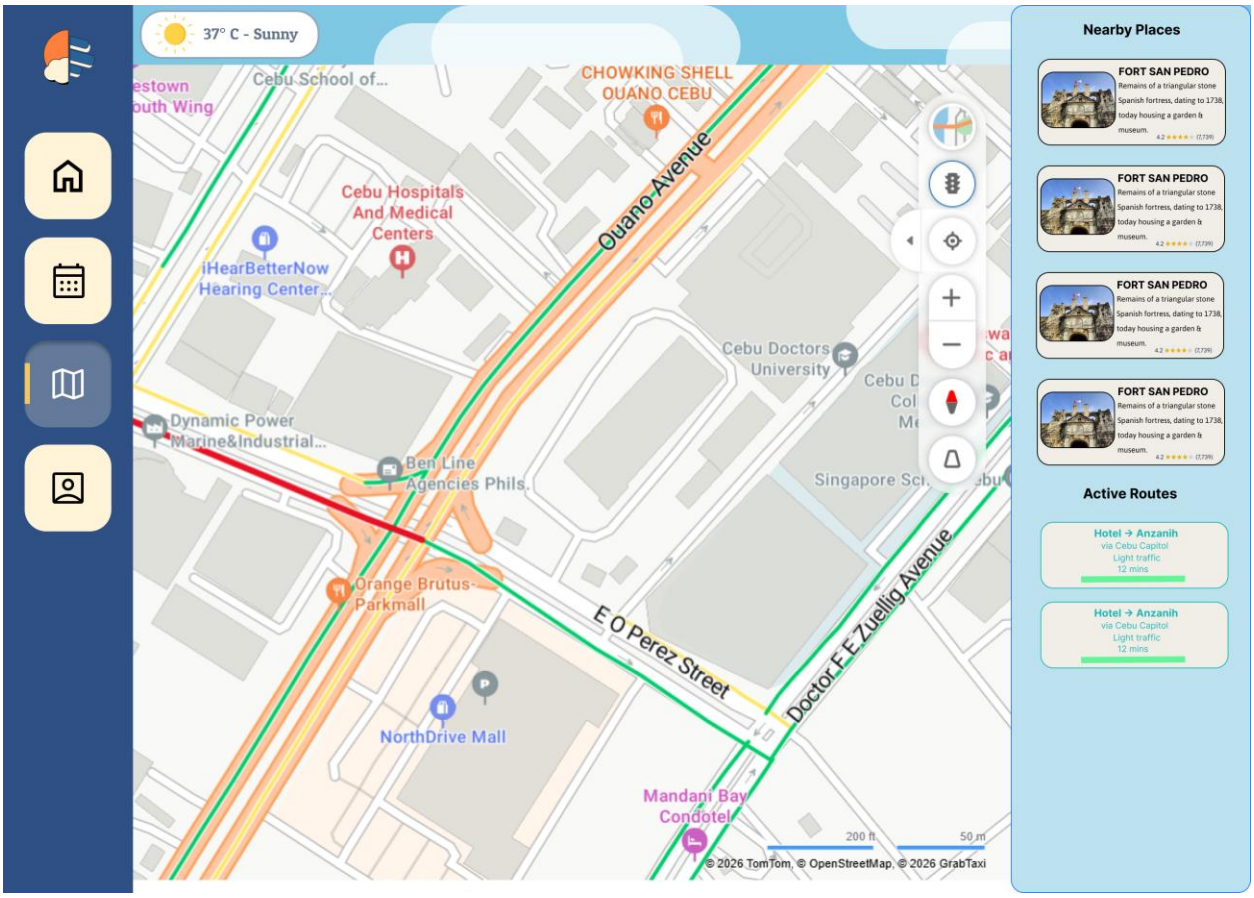
6.1 Overview of User Interface

The ScheduleSkies system provides a web and mobile-based interface that allows users to plan trips, manage schedules, and receive intelligent recommendations. From the user's perspective, the system is task-oriented, guiding them through itinerary creation, optimization, and monitoring.

6.2 Screen Images







6.3 Screen Objects and Actions

The ScheduleSkies interface is composed of four main screens: Home, Events, Map, and Account, each accessible through the navigation sidebar. On the Home screen, users view a summary of their trip, including current weather conditions, daily forecasts, traffic updates, notifications, upcoming plans, and suggested places. Interactive cards allow users to select events for more details, review traffic advisories, read alerts, or add recommended places directly to their itinerary. Visual indicators such as color-coded traffic levels, highlighted notifications, and updated temperature displays provide immediate feedback and real-time system updates.

The Events screen allows users to manage their itinerary. A search bar and category filters help users refine visible events, while Add and Edit buttons enable event creation and modification. Each event card displays the title, time, location, category, weather condition, and status indicators such as Completed or Conflict. If scheduling conflicts occur, the system highlights the issue and presents AI-generated adjustment suggestions. Users can accept recommended changes or manually update event details, with the schedule refreshing instantly and conflict warnings disappearing once resolved.

The Map screen provides a geographic view of itinerary locations. Events are displayed as interactive markers on a map, allowing users to visualize routes, distances, and estimated travel times. Users can select markers to view event details, preview routes between locations, and

review traffic conditions. The system dynamically updates routing information and visually highlights selected paths to provide clear navigation feedback.

The Account screen allows users to manage personal and preference information. Users can update profile details, set transportation preferences, manage notification settings, and review saved trips. Input forms and toggle controls support easy modification of account data. Confirmation messages and visual indicators ensure users are informed when updates are successfully saved. Across all screens, consistent navigation, real-time updates, and immediate visual feedback ensure that each screen object clearly supports its associated user action.

7. REQUIREMENTS MATRIX

The following matrix traces the system components and major data structures to the functional requirements listed in Section 3.1 of the SRS.

Requirement	Home Screen	Events Screen	Map Screen	Account Screen
Smart Schedule & Itinerary Planner		✓		
Weather-Aware Planning	✓	✓		
Traffic & Travel Prediction	✓		✓	
Real-Time Location Awareness	✓		✓	
Smart Place & Destination Discovery	✓		✓	
User Profiles & Personalization				✓
Smart Recommendations	✓	✓	✓	
Notifications and Alert	✓			
Collaboration and Sharing		✓		
Map & Visualization Features			✓	

8. APPENDICES

Figma Link: https://www.figma.com/design/nXTn8u5HBe5w9h3C5dK3qU/ScheduleSkies?node-id=0-1&t=51gtWXucPL0PScS8-1